

Instruções para o Codex - Sistema Gerenciador IPTV

Crie um sistema web completo para gerenciamento e reprodução de listas IPTV, com autenticação de usuários, cadastro de listas .m3u, .m3u8 e URLs remotas, importação de canais, categorização, busca e player integrado.

O sistema deve permitir que o usuário autenticado cadastre uma ou várias listas IPTV, tanto por upload de arquivo quanto por URL. O sistema deve processar o conteúdo dessas listas, extrair os canais, armazenar os dados no banco e disponibilizar uma interface para navegação e reprodução dos canais.

O sistema deve ser pensado para uso com listas IPTV legítimas, próprias ou autorizadas.

1. Stack sugerida

Crie o projeto em arquitetura **monorepo**, contendo:

```
iptv-manager/  
|-- backend/  
|-- frontend/  
|-- docker/  
|-- docs/  
|-- scripts/  
`-- README.md
```

Backend

Use:

```
Java 21  
Spring Boot 3.x  
Spring Security  
JWT  
Spring Data JPA  
PostgreSQL  
Flyway  
Maven
```

Frontend

Use:

```
Angular 18+  
TypeScript  
Angular Router  
Reactive Forms  
Angular Material ou TailwindCSS  
HLS.js para reprodução de streams .m3u8
```

Infraestrutura

Use:

```
Docker  
Docker Compose  
PostgreSQL  
Nginx opcional para servir frontend em produção
```

2. Funcionalidades principais

2.1 Autenticação

O sistema deve possuir autenticação completa com JWT.

Criar as seguintes funcionalidades:

- Cadastro de usuário

Login
Logout no frontend
Validação de token
Proteção de rotas no Angular
Proteção de endpoints no backend
Refresh token, se possível
Controle de perfil de usuário

Perfis iniciais:

ADMIN
USER

Regras:

Usuário ADMIN pode visualizar e gerenciar todos os usuários e listas.
Usuário USER só pode visualizar e gerenciar suas próprias listas.

3. Módulo de usuários

Criar entidade User.

Campos sugeridos:

id: Long
name: String
email: String
password: String
role: Enum USER/ADMIN
active: Boolean
createdAt: LocalDateTime
updatedAt: LocalDateTime

Regras:

Email deve ser único.
Senha deve ser armazenada com BCrypt.
Usuário inativo não pode fazer login.

Endpoints sugeridos:

POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
GET /api/auth/me

GET /api/users
GET /api/users/{id}
PUT /api/users/{id}
DELETE /api/users/{id}

4. Módulo de listas IPTV

O sistema deve permitir cadastrar listas IPTV de duas formas:

1. Upload de arquivo .m3u ou .m3u8
2. Cadastro por URL remota

Criar entidade IptvList.

Campos sugeridos:

id: Long
name: String
description: String
sourceType: Enum FILE/URL
sourceUrl: String
originalFileName: String
status: Enum PENDING/PROCESSING/IMPORTED/ERROR
totalChannels: Integer
lastImportAt: LocalDateTime
errorMessage: String
owner: User
createdAt: LocalDateTime
updatedAt: LocalDateTime

Endpoints sugeridos:

```
POST /api/iptv-lists/upload
POST /api/iptv-lists/url
GET /api/iptv-lists
GET /api/iptv-lists/{id}
PUT /api/iptv-lists/{id}
DELETE /api/iptv-lists/{id}
POST /api/iptv-lists/{id}/import
POST /api/iptv-lists/{id}/refresh
```

Regras:

Ao cadastrar uma lista por upload, salvar o arquivo no diretório configurado.
Ao cadastrar uma lista por URL, salvar a URL e permitir importação posterior.
Ao importar uma lista, ler o conteúdo e extrair os canais.
Se a lista já tiver canais cadastrados, permitir limpar e reimportar.
Cada lista pertence a um usuário.
Usuário comum só acessa suas próprias listas.
ADMIN pode acessar todas.

5. Parser de listas M3U/M3U8

Implementar um serviço chamado:

M3uParserService

Ele deve ser capaz de processar listas nesse formato:

```
#EXTM3U
#EXTINF:-1 tvg-id="canal1" tvg-name="Canal 1" tvg-logo="https://logo.com/canal1.png"
    group-title="Filmes",Canal 1
http://servidor.com/live/canal1.m3u8
#EXTINF:-1 tvg-id="canal2" tvg-name="Canal 2" tvg-logo="https://logo.com/canal2.png"
    group-title="Esportes",Canal 2
http://servidor.com/live/canal2.m3u8
```

Extrair os seguintes dados:

Nome do canal
URL do stream
Grupo ou categoria
Logo
tvg-id
tvg-name
tvg-logo
group-title
Duração, se existir

Criar uma classe DTO:

ParsedChannelDTO

Campos:

```
name: String
streamUrl: String
groupTitle: String
logoUrl: String
tvgId: String
tvgName: String
duration: String
rawExtinf: String
```

Regras do parser:

Ignorar linhas vazias.
Ignorar comentários não relevantes.
Para cada linha #EXTINF, a próxima linha válida deve ser considerada a URL do canal.
Se não houver grupo, usar "Sem categoria".
Se não houver logo, deixar nulo.
Se não houver nome, tentar usar tvg-name.
Se ainda assim não houver nome, usar "Canal sem nome".
Validar se a URL começa com http:// ou https://.
Evitar duplicidade dentro da mesma lista usando streamUrl como referência.

6. Módulo de canais

Criar entidade Channel.

Campos sugeridos:

```
id: Long
name: String
streamUrl: String
groupTitle: String
logoUrl: String
tvId: String
tvName: String
duration: String
favorite: Boolean
active: Boolean
iptvList: IptvList
owner: User
createdAt: LocalDateTime
updatedAt: LocalDateTime
```

Endpoints sugeridos:

```
GET    /api/channels
GET    /api/channels/{id}
GET    /api/iptv-lists/{listId}/channels
PUT    /api/channels/{id}
DELETE /api/channels/{id}

GET    /api/channels/search?term=
GET    /api/channels/groups
GET    /api/channels/group/{groupName}

POST   /api/channels/{id}/favorite
DELETE /api/channels/{id}/favorite
GET    /api/channels/favorites
```

Filtros necessários:

```
Buscar por nome
Filtrar por lista
Filtrar por categoria
Filtrar favoritos
Filtrar ativos/inativos
Paginação
Ordenação alfabética
```

7. Player IPTV

No frontend, criar uma tela de reprodução de canal.

Usar:

```
HLS.js
HTML5 video
```

A tela deve conter:

```
Player de vídeo
Nome do canal
Logo do canal
Categoria
Botão favorito
Botão voltar
Lista lateral de canais
Busca rápida
```

Comportamento:

```
Se o navegador suportar HLS nativamente, usar o player HTML5 diretamente.
Se não suportar, usar HLS.js.
Exibir mensagem amigável se o stream não carregar.
Exibir loading durante carregamento.
Permitir trocar de canal sem sair da tela.
```

Exemplo de componente Angular esperado:

PlayerComponent

Rotas sugeridas:

```
/login
/register
/dashboard
/lists
/lists/new
/lists/:id
/channels
/channels/:id/play
/favorites
/admin/users
```

8. Frontend

Criar um frontend completo em Angular.

Telas obrigatórias

```
Tela de login
Tela de cadastro
Dashboard
Listagem de listas IPTV
Cadastro de lista por URL
Upload de arquivo .m3u/.m3u8
Detalhes da lista
Listagem de canais
Busca de canais
Filtro por categoria
Tela de favoritos
Player de canal
Área administrativa de usuários
```

Layout

Criar layout com:

```
Menu lateral
Topo com usuário logado
Botão de logout
Responsividade para desktop e mobile
Cards de canais
Cards de listas
Tabela de listas
Tabela de usuários
```

9. Dashboard

O dashboard deve exibir:

```
Total de listas cadastradas
Total de canais importados
Total de categorias
Total de favoritos
Últimas listas importadas
Últimos canais adicionados
```

Endpoint sugerido:

```
GET /api/dashboard
```

DTO sugerido:

```
DashboardDTO {
  totalLists: Long
  totalChannels: Long
  totalGroups: Long
  totalFavorites: Long
  recentLists: List<IptvListDTO>
  recentChannels: List<ChannelDTO>
}
```

10. Banco de dados

Criar migrations com Flyway.

Tabelas principais:

```
users
iptv_lists
channels
refresh_tokens
```

Script inicial sugerido

Criar migration:

```
V1__create_users_table.sql
V2__create_iptv_lists_table.sql
V3__create_channels_table.sql
V4__create_refresh_tokens_table.sql
```

Estrutura esperada

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(30) NOT NULL,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP
);

CREATE TABLE iptv_lists (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  description TEXT,
  source_type VARCHAR(20) NOT NULL,
  source_url TEXT,
  original_file_name VARCHAR(255),
  status VARCHAR(30) NOT NULL,
  total_channels INTEGER DEFAULT 0,
  last_import_at TIMESTAMP,
  error_message TEXT,
  owner_id BIGINT NOT NULL,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP,
  CONSTRAINT fk_iptv_lists_owner
    FOREIGN KEY (owner_id) REFERENCES users(id)
);

CREATE TABLE channels (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  stream_url TEXT NOT NULL,
  group_title VARCHAR(255),
  logo_url TEXT,
  tv_id VARCHAR(255),
  tv_name VARCHAR(255),
  duration VARCHAR(50),
  favorite BOOLEAN NOT NULL DEFAULT FALSE,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  iptv_list_id BIGINT NOT NULL,
  owner_id BIGINT NOT NULL,
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP,
  CONSTRAINT fk_channels_iptv_list
    FOREIGN KEY (iptv_list_id) REFERENCES iptv_lists(id),
  CONSTRAINT fk_channels_owner
    FOREIGN KEY (owner_id) REFERENCES users(id)
);

CREATE INDEX idx_channels_name ON channels(name);
CREATE INDEX idx_channels_group_title ON channels(group_title);
CREATE INDEX idx_channels_iptv_list_id ON channels(iptv_list_id);
CREATE INDEX idx_channels_owner_id ON channels(owner_id);
CREATE INDEX idx_iptv_lists_owner_id ON iptv_lists(owner_id);
```

11. Segurança

Implementar segurança com Spring Security.

Regras:

- Endpoints `/api/auth/**` devem ser públicos.
- Todos os demais endpoints devem exigir JWT.
- Rotas administrativas exigem perfil ADMIN.
- Usuários comuns só acessam seus próprios dados.
- Senha deve ser criptografada com BCrypt.
- JWT deve ter expiração configurável.
- CORS deve permitir o frontend.

Configurações esperadas no backend:

```
app.jwt.secret=${JWT_SECRET}
app.jwt.expiration-ms=3600000
app.upload.dir=uploads
spring.servlet.multipart.max-file-size=20MB
spring.servlet.multipart.max-request-size=20MB
```

Não deixar secret hardcoded no código.

12. Validações

Implementar validações no backend e frontend.

Usuário

- Nome obrigatório
- Email obrigatório e válido
- Senha obrigatória com no mínimo 6 caracteres
- Email único

Lista IPTV

- Nome obrigatório
- Arquivo obrigatório no upload
- Arquivo deve ser `.m3u` ou `.m3u8`
- URL obrigatória no cadastro por URL
- URL deve começar com `http://` ou `https://`

Canal

- Nome obrigatório
- URL do stream obrigatória
- URL deve começar com `http://` ou `https://`

13. Tratamento de erros

Criar tratamento global de erros no backend com:

```
@RestControllerAdvice
```

Retornar erros padronizados:

```
{
  "timestamp": "2026-06-02T10:00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Mensagem amigável",
  "path": "/api/..."
}
```

Tratar:

- Erro de autenticação
- Token inválido
- Acesso negado
- Recurso não encontrado
- Arquivo inválido
- Erro ao importar lista

Erro de validação
Erro interno

14. Importação de listas

A importação deve seguir o seguinte fluxo:

1. Usuário cadastra lista por upload ou URL.
2. Sistema cria registro com status PENDING.
3. Usuário aciona importação ou o sistema importa automaticamente.
4. Status muda para PROCESSING.
5. Sistema lê o conteúdo.
6. Parser extrai canais.
7. Sistema salva canais no banco.
8. Atualiza totalChannels.
9. Atualiza lastImportAt.
10. Status muda para IMPORTED.
11. Se houver erro, status muda para ERROR e salva errorMessage.

Implementar importação em serviço separado:

IptvImportService

Métodos esperados:

```
importFromFile(Long listId, User currentUser)
importFromUrl(Long listId, User currentUser)
refreshList(Long listId, User currentUser)
```

15. Atualização de listas remotas

Para listas cadastradas por URL, criar funcionalidade de atualizar.

Regras:

- Ao atualizar, baixar novamente o conteúdo da URL.
- Remover canais antigos da lista.
- Importar os canais novamente.
- Atualizar total de canais.
- Manter a mesma lista.
- Preservar favoritos se possível, comparando pela streamUrl.

16. Performance

O sistema deve considerar listas grandes, com milhares de canais.

Implementar:

- Paginação em todas as listagens de canais
- Busca por nome usando índice no banco
- Importação em lote quando possível
- Evitar salvar canal um por um se puder usar batch
- Não carregar todos os canais no frontend de uma vez
- Limitar tamanho máximo do arquivo

Configurações JPA sugeridas:

```
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

17. Docker Compose

Criar docker-compose.yml com:

- PostgreSQL
- Backend Spring Boot
- Frontend Angular/Nginx

Variáveis esperadas:

POSTGRES_DB=iptvdb


```
POSTGRES_USER=iptv
POSTGRES_PASSWORD=iptv123
JWT_SECRET=trocar_essa_chave_em_producao
```

18. README

Criar um README.md completo contendo:

```
Descrição do projeto
Tecnologias usadas
Como rodar com Docker
Como rodar backend local
Como rodar frontend local
Como configurar banco
Como importar listas IPTV
Exemplos de arquivo .m3u
Endpoints principais
Usuário administrador inicial
Observações de segurança
```

19. Dados iniciais

Criar uma migration ou seed para usuário administrador inicial.

Exemplo:

```
Email: admin@admin.com
Senha: admin123
Role: ADMIN
```

A senha deve ser salva criptografada com BCrypt.

Se preferir, criar o admin inicial no startup da aplicação usando CommandLineRunner, lendo email e senha de variáveis de ambiente:

```
ADMIN_EMAIL=admin@admin.com
ADMIN_PASSWORD=admin123
```

20. Estrutura esperada do backend

Criar pacotes organizados assim:

```
com.example.iptvmanager
|-- config
|-- security
|-- auth
|-- user
|-- iptvlist
|-- channel
|-- dashboard
|-- parser
|-- exception
-- common
```

Dentro de cada módulo, usar estrutura:

```
controller
service
repository
dto
entity
mapper
```

21. Estrutura esperada do frontend

Criar estrutura Angular assim:

```
src/app/
|-- core/
|   |-- auth/
|   |-- guards/
```

```
| |-- interceptors/  
| |-- services/  
|-- shared/  
| |-- components/  
| |-- models/  
|-- features/  
| |-- auth/  
| |-- dashboard/  
| |-- iptv-lists/  
| |-- channels/  
| |-- player/  
| |-- admin/  
|-- layout/
```

22. Requisitos importantes do player

O player deve funcionar com streams HLS.

Instalar no frontend:

```
npm install hls.js
```

Criar serviço ou componente que faça:

```
Detectar se o navegador suporta HLS nativamente.  
Caso suporte, atribuir diretamente a URL ao video.src.  
Caso não suporte, inicializar HLS.js.  
Destruir instância anterior do HLS ao trocar de canal.  
Tratar erro de carregamento.
```

23. Exemplo de regra para reprodução

Ao clicar em um canal:

```
Frontend chama GET /api/channels/{id}  
Backend valida se o canal pertence ao usuário logado  
Frontend abre /channels/{id}/play  
Player carrega streamUrl
```

Não expor canais de outros usuários.

24. Testes

Criar testes para:

```
Parser de arquivo M3U  
Autenticação  
Cadastro de lista  
Importação de lista  
Busca de canais  
Permissões por usuário
```

Testes mínimos:

```
M3uParserServiceTest  
AuthServiceTest  
IptvImportServiceTest  
ChannelControllerTest
```

25. Critérios de aceite

O sistema será considerado pronto quando:

```
Usuário consegue se cadastrar.  
Usuário consegue fazer login.  
Usuário consegue cadastrar uma lista por URL.  
Usuário consegue fazer upload de arquivo .m3u ou .m3u8.  
Sistema consegue importar canais da lista.  
Sistema mostra os canais importados.  
Sistema separa canais por categoria.  
Usuário consegue buscar canais pelo nome.
```

Usuário consegue favoritar canais.
Usuário consegue reproduzir canal .m3u8 no player.
Usuário comum não consegue acessar lista de outro usuário.
ADMIN consegue visualizar todos os usuários e listas.
Backend roda com PostgreSQL.
Frontend consome corretamente a API.
Projeto roda com Docker Compose.
README explica como instalar e executar.

26. Observações legais e de segurança

Adicionar no README:

Este sistema é apenas um gerenciador/reprodutor de listas IPTV.
Ele não fornece canais, conteúdo, streams ou listas.
O usuário é responsável por adicionar apenas listas e conteúdos que tenha direito de acessar.
O sistema não deve burlar autenticação, DRM, paywall ou qualquer mecanismo de proteção.

27. Entregáveis esperados

Ao finalizar, entregar:

Código completo do backend
Código completo do frontend
Migrations Flyway
Docker Compose
README
Exemplo de arquivo .m3u para teste
Collection Postman ou arquivo HTTP para testar endpoints

28. Exemplo de arquivo `.m3u` para testes

Criar em:

docs/examples/sample.m3u

Conteúdo:

```
#EXTM3U
#EXTINF:-1 tvg-id="canal-news" tvg-name="Canal News" tvg-logo="https://example.com/news.png"
  group-title="Notícias",Canal News
https://example.com/stream/news.m3u8
#EXTINF:-1 tvg-id="canal-sports" tvg-name="Canal Sports"
  tvg-logo="https://example.com/sports.png" group-title="Esportes",Canal Sports
https://example.com/stream/sports.m3u8
#EXTINF:-1 tvg-id="canal-movies" tvg-name="Canal Movies"
  tvg-logo="https://example.com/movies.png" group-title="Filmes",Canal Movies
https://example.com/stream/movies.m3u8
```

29. Resultado esperado

Crie o projeto completo, funcional e organizado, com backend e frontend integrados. Priorize código limpo, boas práticas, segurança, separação de responsabilidades e facilidade para manutenção futura.

Não entregue apenas exemplos parciais. Gere todos os arquivos necessários para o projeto rodar localmente e via Docker.